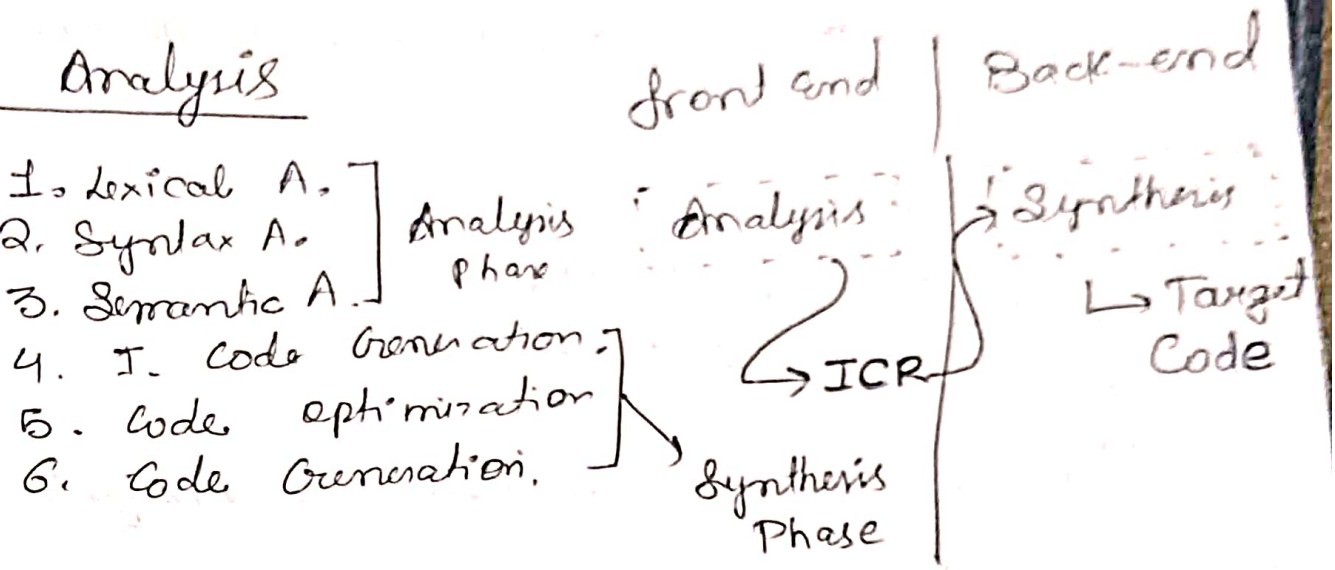
C||W
21|1|26

Compiler → Source lang. to target lang. 6 Phases

Lexical Analysis



Lexical Analysis → consider code as a source of character - character Stream

↳ Remove useless things
  ↳ Space, tab, newline
  ↳ Comments
  ↳ expand all macros.
  ↳

Lexer → program which does lexical Analysis. and generates stream of characters to form meaningful ~~chara~~ → grouping to generate tokens.

↳ lexical errors
  ↳ unrecognized token
  ↳ misspellings
  ↳ out of bounds

⇒ how to generate valid tokens

↳ Symbol table will be covered to all the phases

↳ this is output of Lexical Analysis phase

Syntax Analysis

↳ generates parse tree

↳ may change Symbol table

↳ Passes these two to Semantic Analyser

Eg:-

A = B + C * 100 → LA

Token → whitespace / newline

Constants

identifiers

Special Symbols

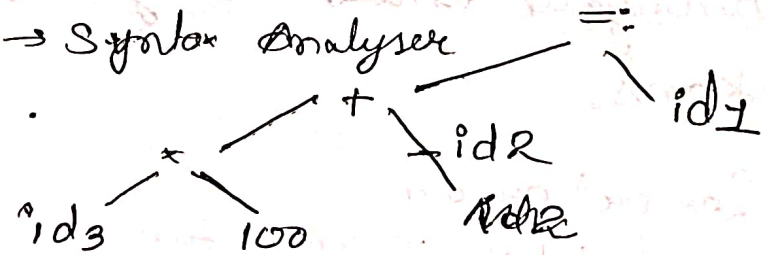
↳ operators

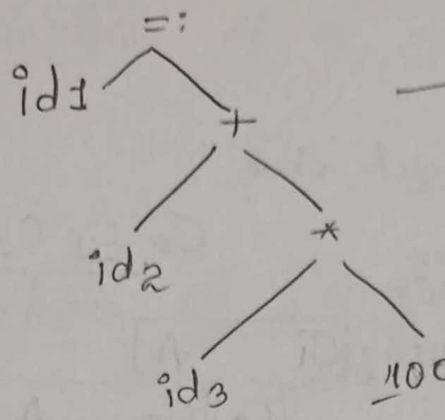
↳ Lexim with a meaningful

id1 = id2 + id3 * 100

↓

SA → Syntax Analyser





→ output of Syntax Analyser

↳ Annotated Parse tree will be output of Semantic Analyser

A: P-T → ICG → ToC → Code Generation

↓
Intermediate code generation.

↳

```
temp1 = int_to_real(100);
temp2 = id3 x temp1;
temp3 = id2 + temp2;
id1 = temp3;
```

```
temp1 = id3 x int_to_real(100);
id1 = id2 + temp1
```

optimised code

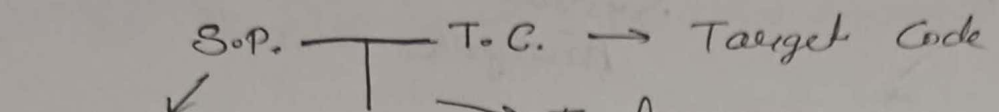
↓

Code Generation

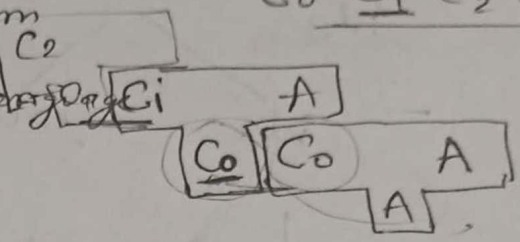
↳

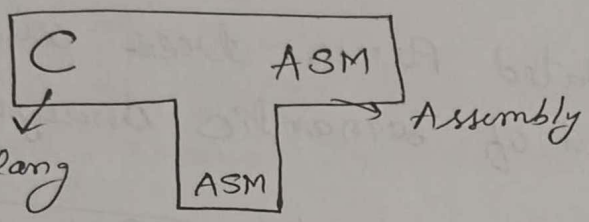
```
MOVF   #100   R1
MOVF   id3    R2
MULF   R2     R1
MOVF   id2    R3
ADDF   R3     R1
MOVF   R1     id1
```

Compiler



C_0 C_1 C_2 C_3

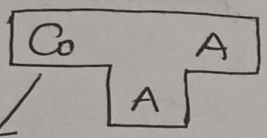




Tea diagram

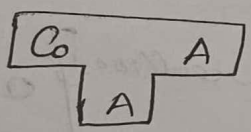
Bootstrapping Compiler

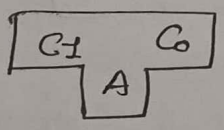
2 marks question on Bootstrapping

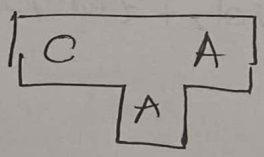
1 

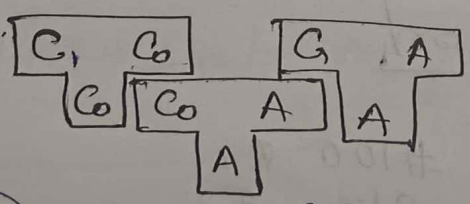
C_0 is a language with very limited functionality.

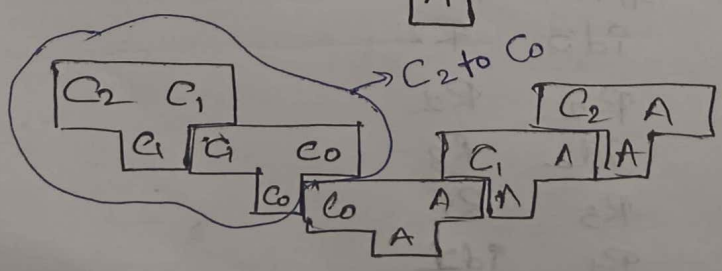
2 Incremented version

C_1 

3. C_2 



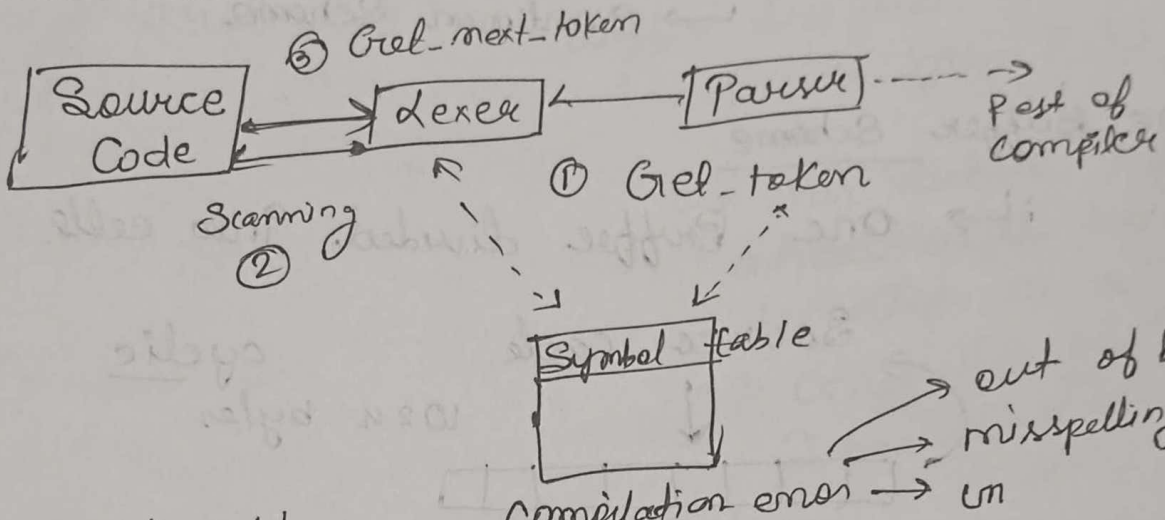




Architecture of a Lexical Analyser

C|W
28|8|20

Structure of Compiler



Symbol table

Name	type	Scope	Memory Address	Value	Information
$ Area function3 (int a)	Area_fn	function	Global	A	—
int area = PI * a * a;	int	Keyword	Global	—	—
return area;	or	id1	Local	A	0
	area	id2	Local	A	0
	PI	constant	Global	A	3.14
	or return	Keyword	Global	—	—

other info

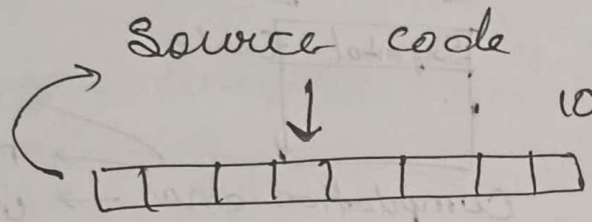
function → argument and return type

lexim generation

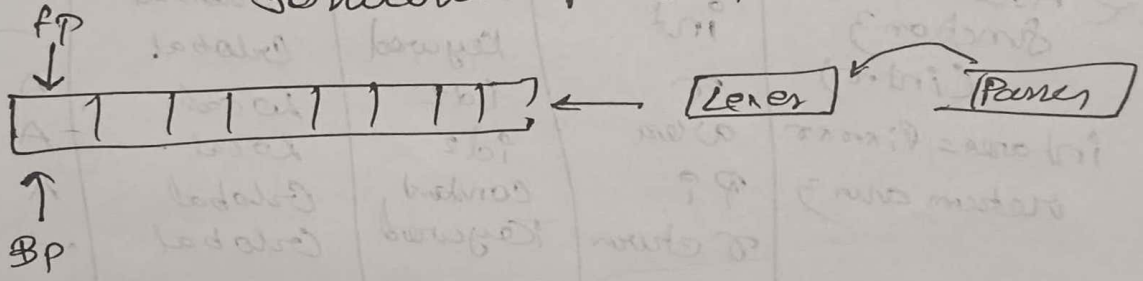
↳ Two types
  ↳ one Buffer Scheme
  Two Buffer scheme
    ↳ Sentinen Scheme.

one Buffer Scheme

↳ one Buffer divided into cells.

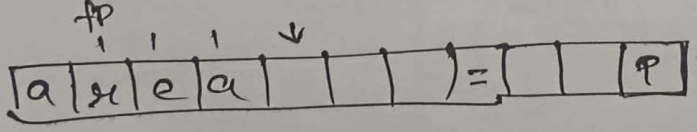


↳ two pointers
  ↳ beginning pointer and forward pointer.



forward pointer moves to check the end of lexim.

Beginning Pointer points to front of lexim



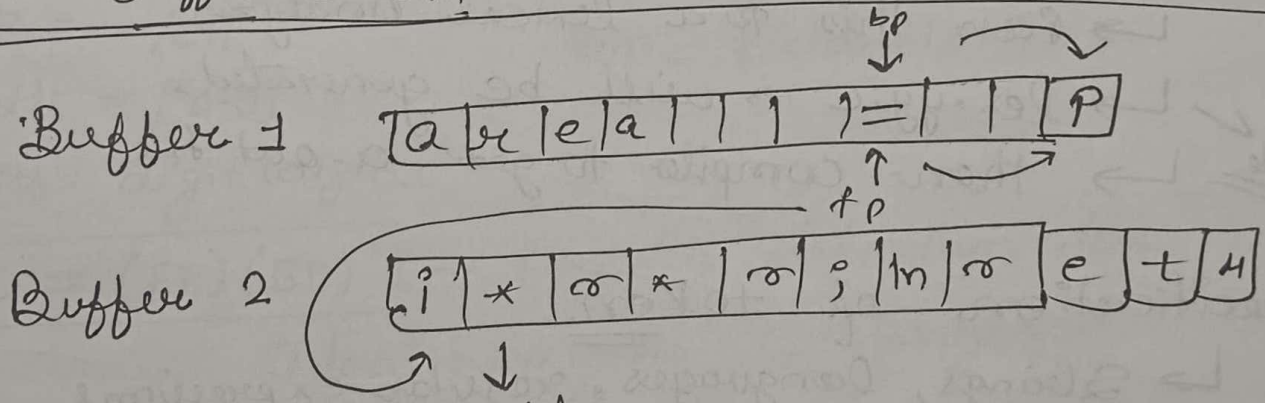
↳ Space aaya then
  aaeen as a lexim
  ↳ then ignore all whitespace

to reach a character
↳ to start checking for another lexim

area, = , P ↳ Buffer size full
↳ Pi ke agla P hi
data
↳ this Partial information will be treated as a lexim
↳ and this is a Drawback

Drawback of one Buffer Scheme
↳ ① unrecognised lexim may be generated
② overflow may be

Two Buffer Scheme



valid termination
↳ So lexim generated = Pi

Pi, *, a, *, a, ; in , retu
↓ ↳ ignore
delimiter
ignore as for termination

↳ error signal generated (out of bounds) then first buffer is used again

Buffer 1 | x | n | | | a | r | e | a | ; |

How to handle Comments

Regex revise
Construct RA
↳ RA
make Calculator

C|W
3|2|26

lex tool → lexical Analyzer

for each type of token we create a pattern.
↳ .l file
↳ pass this to a lexical Analyzer
↳ lex.yy.c → will be generated.
C code ↳ then compile to get a.out file.

Specification of token.
↳ Strings, languages, regular expressions, All thing of TOC.
↳ Pattern will be defined with the help of
~~#~~ Regular definition will be defined over some given Σ which is as follows:-

$d_1 \rightarrow r_1$
$d_2 \rightarrow r_2$
$d_3 \rightarrow r_3$
:
$d_n \rightarrow r_n$

i) Where each d_i is a distinct Symbol. such that $d_i \notin \Sigma$

ii) $\forall r_i$'s are regular expressions
$\hookrightarrow r_i = \Sigma \cup (d_i - d_{i-1})$

iii) $\forall d_i$ are also regular expression Such that d_i defined over $\Sigma \cup (r_i - r_{i-1})$

#Identifiers in C

$\Sigma = \{a\text{-}z, A\text{-}Z, -, 0\text{-}9\}$
$\Sigma = [a\text{-}z] \cup [A\text{-}Z] \cup [-] \cup [0,9]$
$\Sigma = [a|..|z|A|..|z|-|0..|9]$

$LC = [a|..z] = [a\text{-}z]$ ~~FP~~ $\neq$ ~~/CH~~
$UC = [A\text{-}Z]$
CH $\neq$ LC/UC
↳ character
FP = -/CH
SP = FP/digits
digits = [0-9]
digits = digit (digit)*

$\boxed{Id = (FP)(SP)^*}$ → Regular definition for Identifiers in C language.

Operators in C

Op = [+, *, -, /, %, ^]
relop → (< | > | <= | >= | <> | ==)
whitespaces → delim (delim)*
delim → \t | \w | \n
↓
whitespace

Comments

Comment → //(AllChar)* | /* (Sp)** /

AllChar → UC | LC | SSpecialSymbols

Keywords

↳ IF | ELSE | float | int |

Numbers

↳ integer
↳ float
↳ Scientific Notation.

↳ no is compulsory
fraction - optional, Exponent - optional

digit → [0-9]

~~digits → no~~ digits → digit (digit)*

OF → (. digits) | ∈

↳ (. digits)?

Question mark closure

OE → (E (+|-|∈) digits)?

↓

(E (+|-)?) digits)?

Num → digits . OF . OE

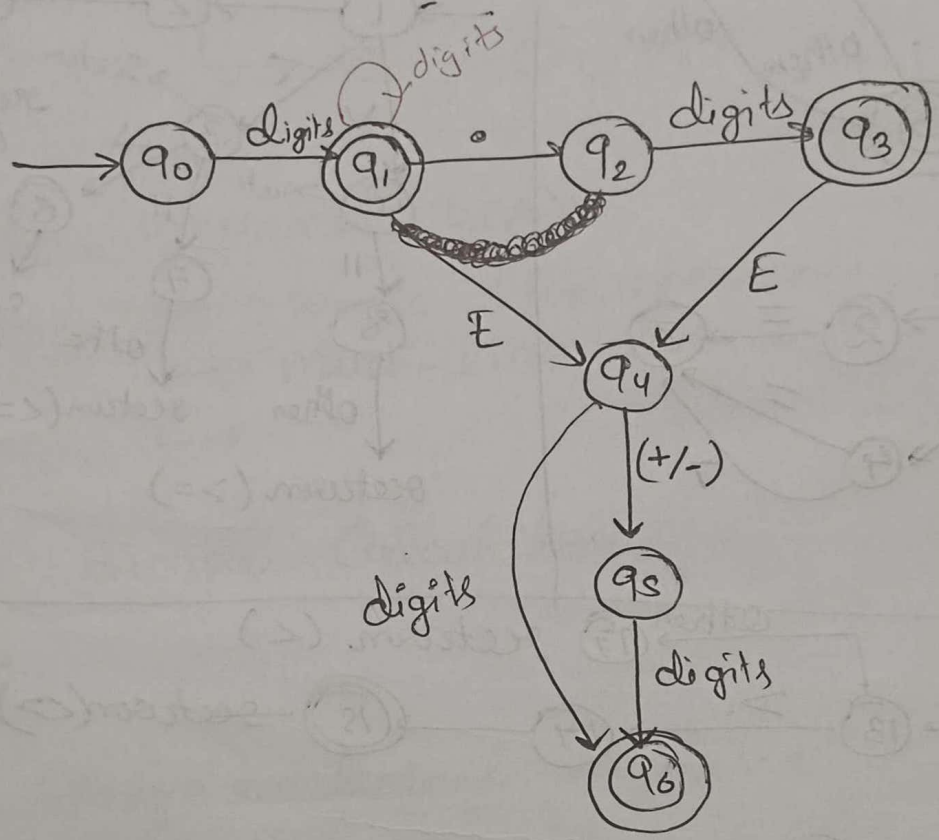
↓

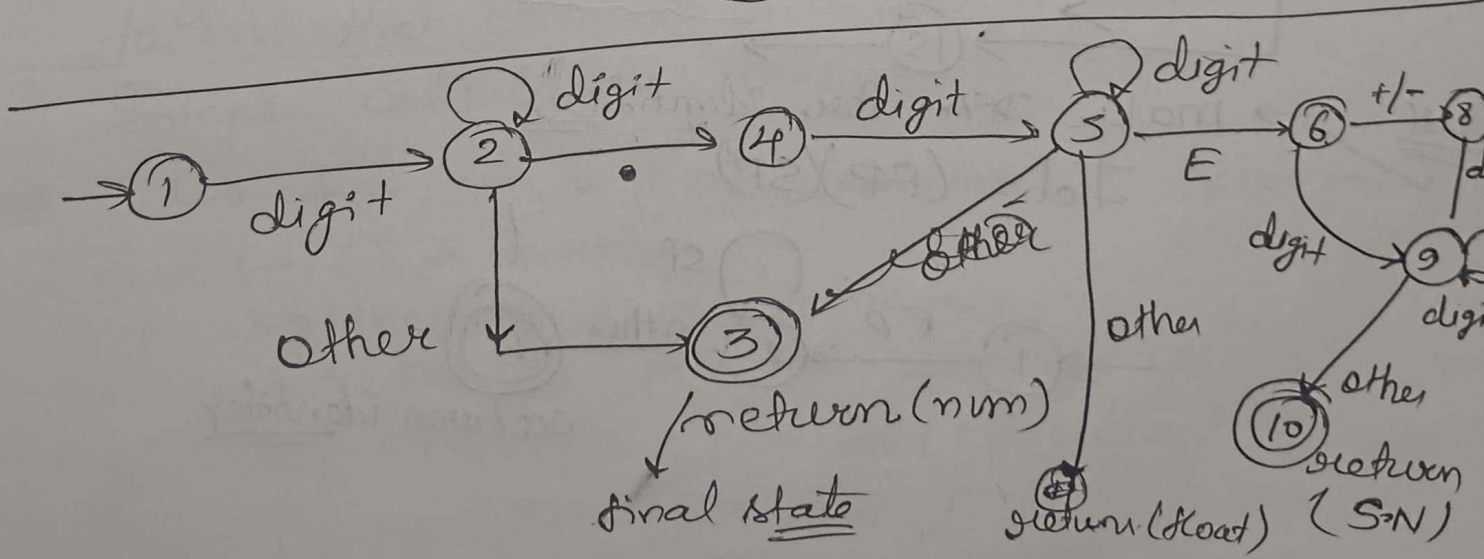
concatenation

Ques Write regular definition for number with using OF and OE

⇒ Num → digits • (• digits)? • (E(+/-)? digits)?

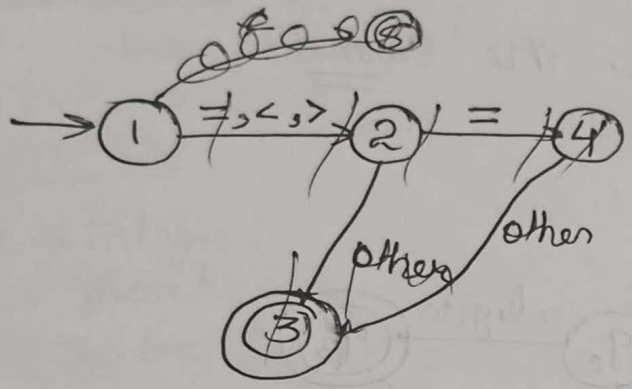
Ques design DFA for the same

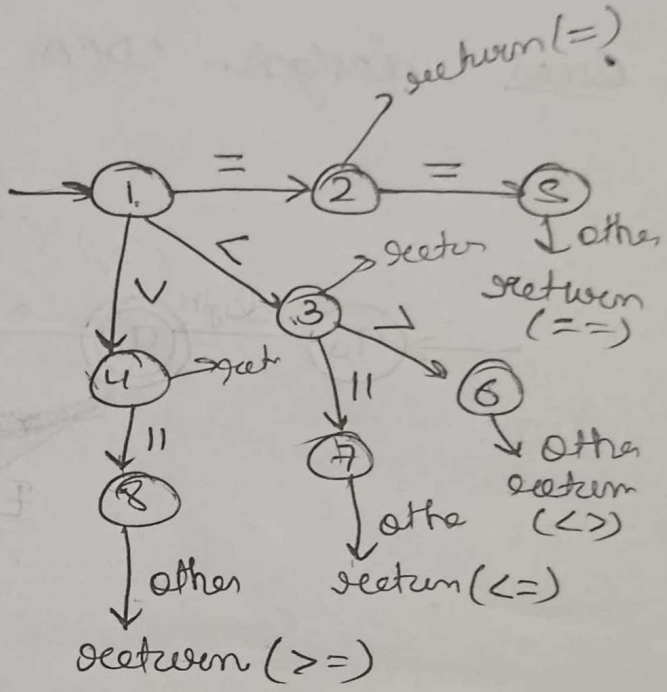


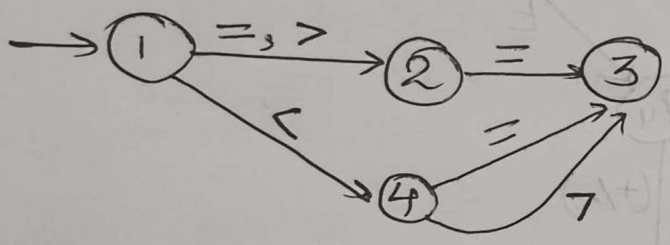


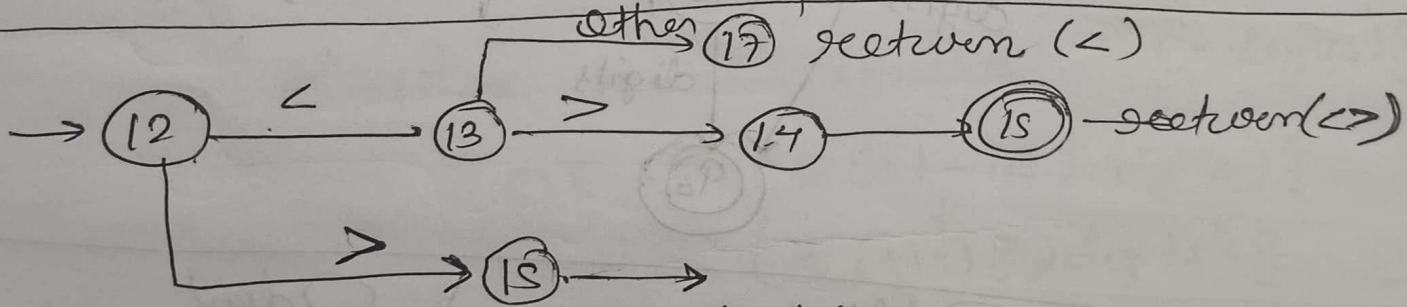
Design Regexp to check whether a DFA given lexeme is relational operator or not

⇒ ROP = {==, <=, >=, >, <, <>}



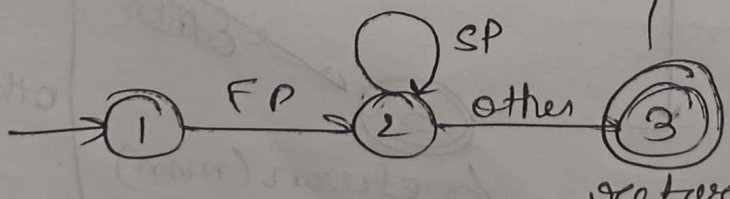






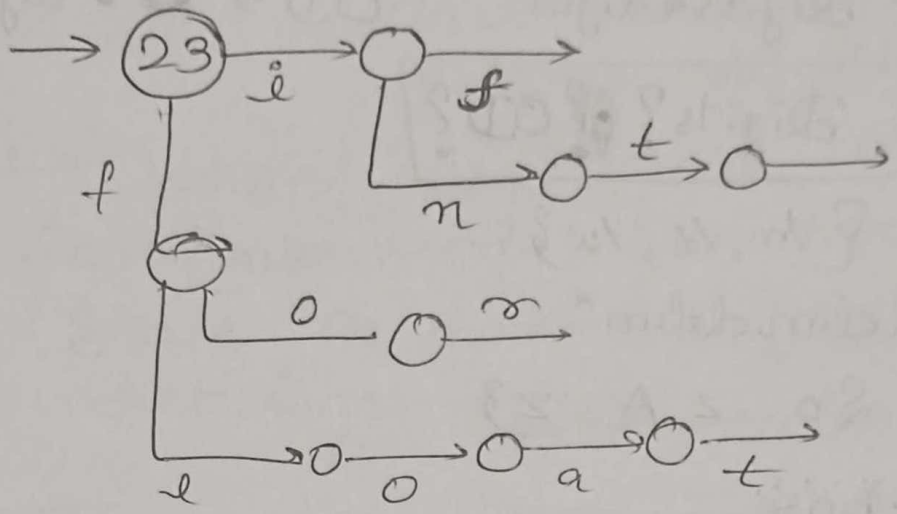
Qual → make DFA for identifier

Id = (FP)(SP)*



Ques Whitespaces (DFA)

Ques DFA for keywords (list out all the keywords)



Ques Comments (DFA)

↳ Single line comments
↳ multi-line comments
↳

Ques Simple Calculator.

C/W
4/2/26

Ques Design a lexical Analyzer to detect all the tokens that can be generated from a Simple calculator program.

⇒ ↳ +, -, /, *, := → assignment operators
↳ () → operator
↳ there can be comments.
↳ handle all the whitespaces
↳ identifier (character followed by character or digit)
↳ numbers → (integer or float)

float can start directly from point

Dig → {0, 1, 2, ... 9}

Digits → Digits digits* OD → . digits

No → Digits? OD?

delim → {\n, \t, \w}

WS → delim delim*

char → {a..z, A..Z}

~~Id → char (AllSym)*~~

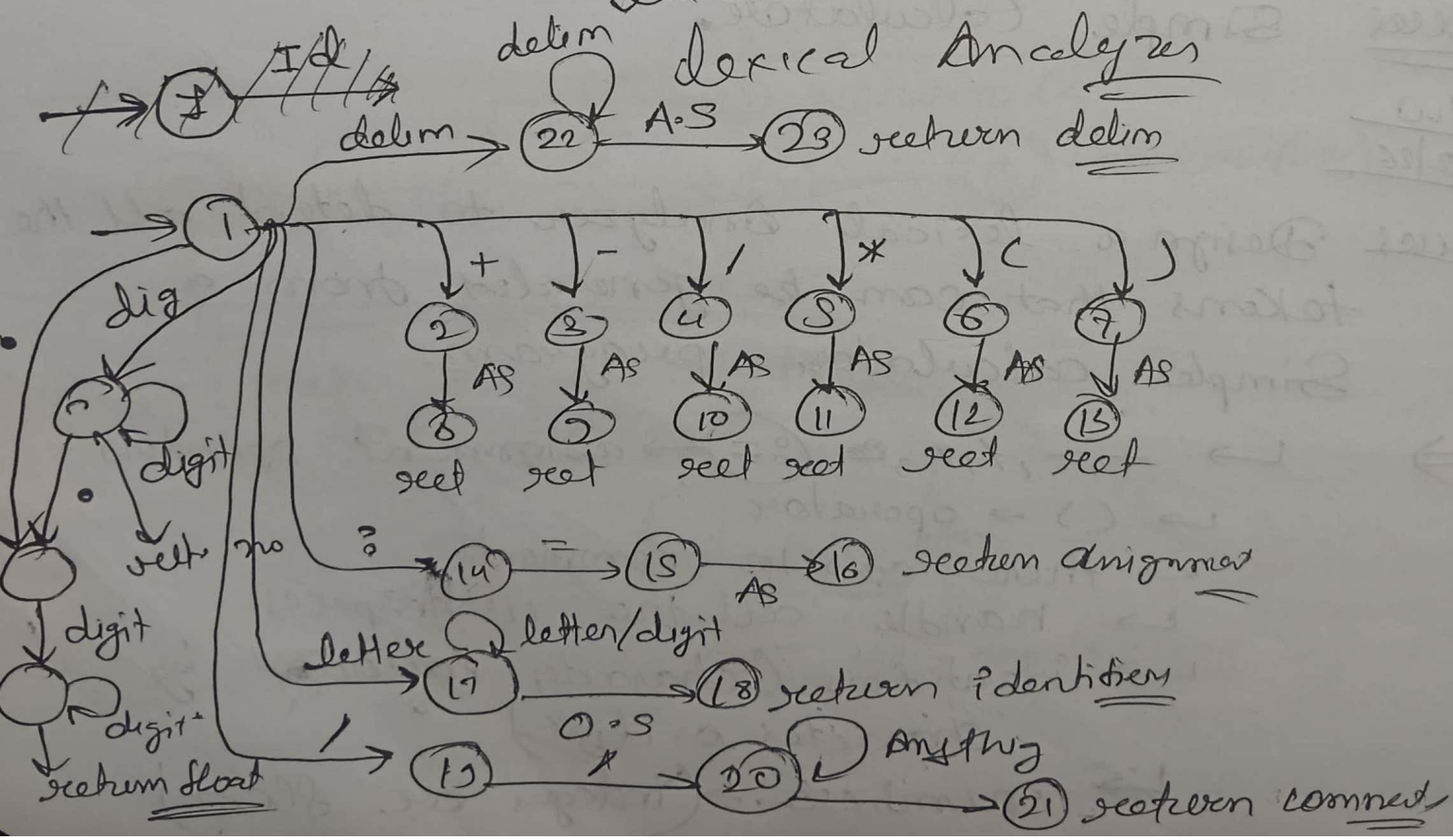
AllSym → {char, digit}

Id → char (AllSym)* Anything → { }

Comments → //(Anything)* | /*(Anything)+*/

lexeme

lexical Analyzer

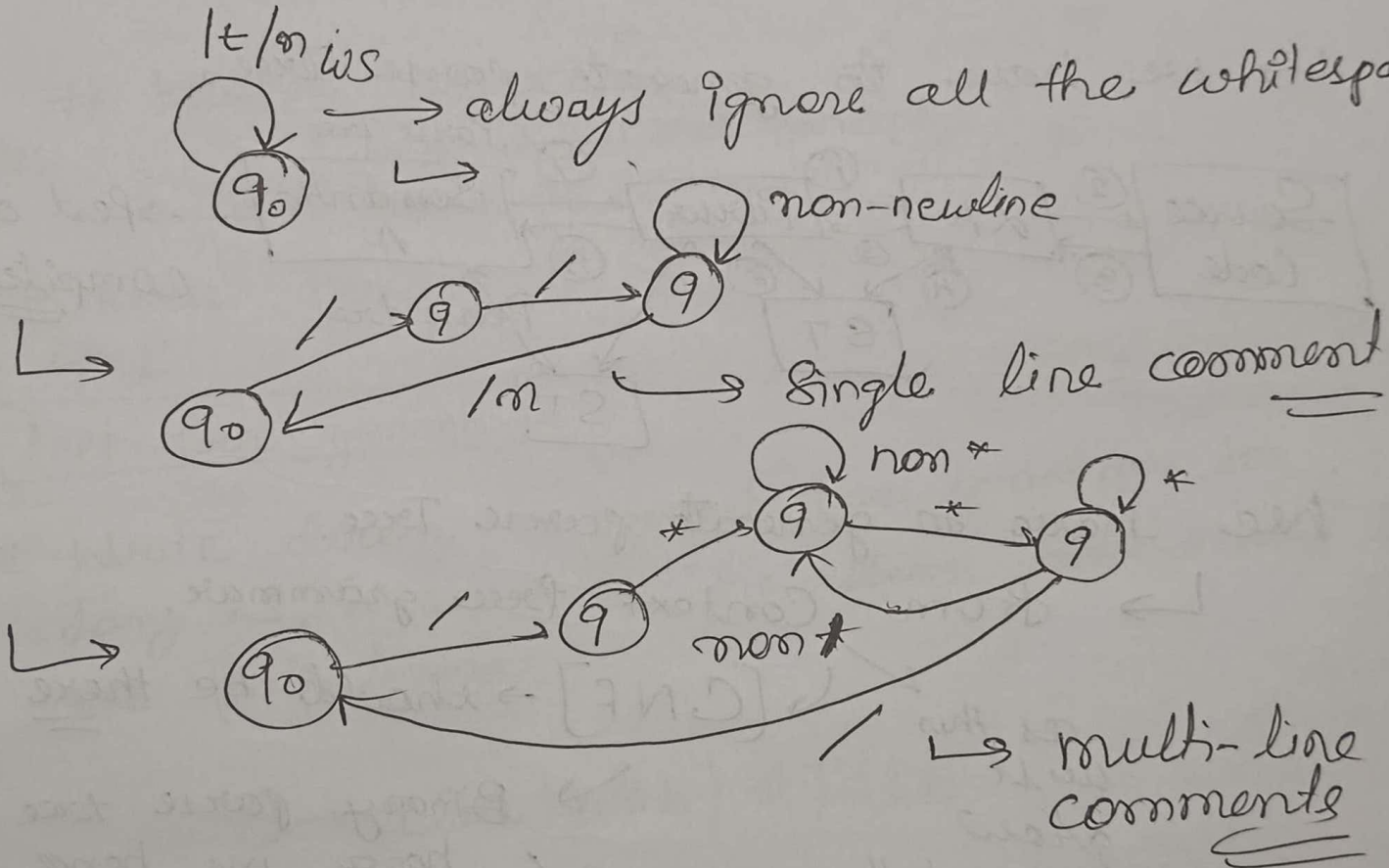


Lexical Analyzer ignores whitespaces and comments

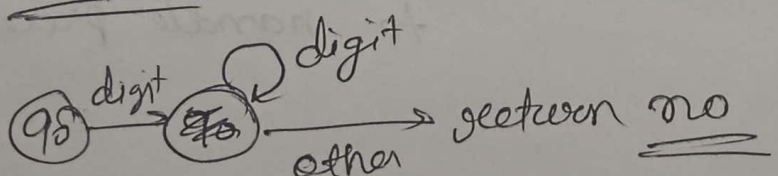
↳ So only recod whitespaces and comments but don't return anything

↳ When we recad whitespaces or Comments, send them back to first State so that we can start reading again

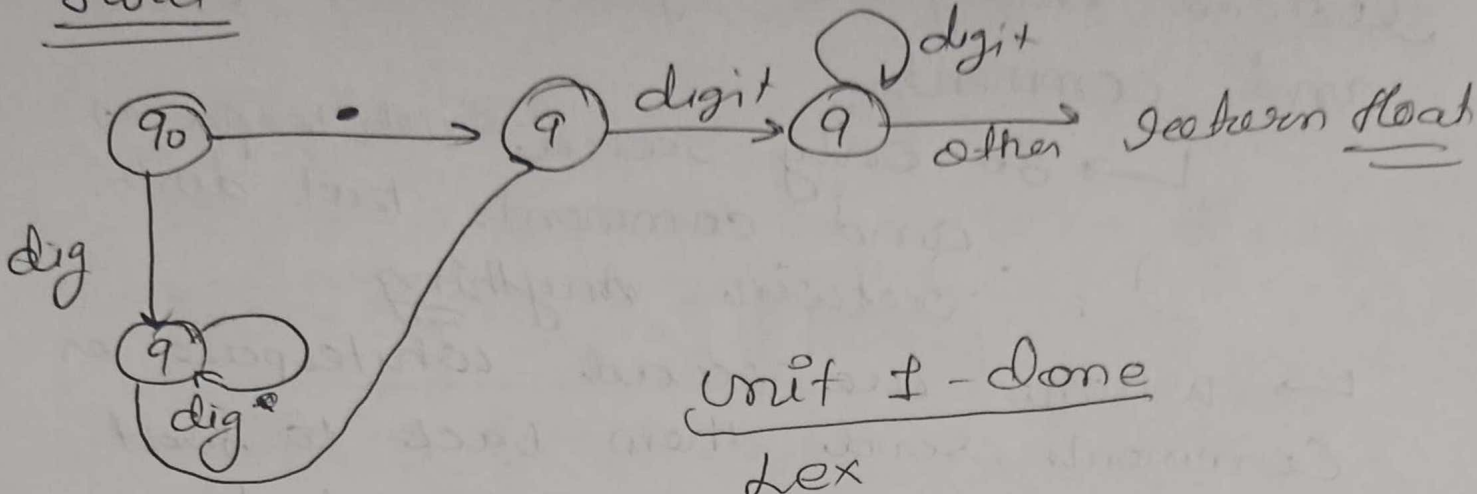
↳



Numbers



float



Unit I - done

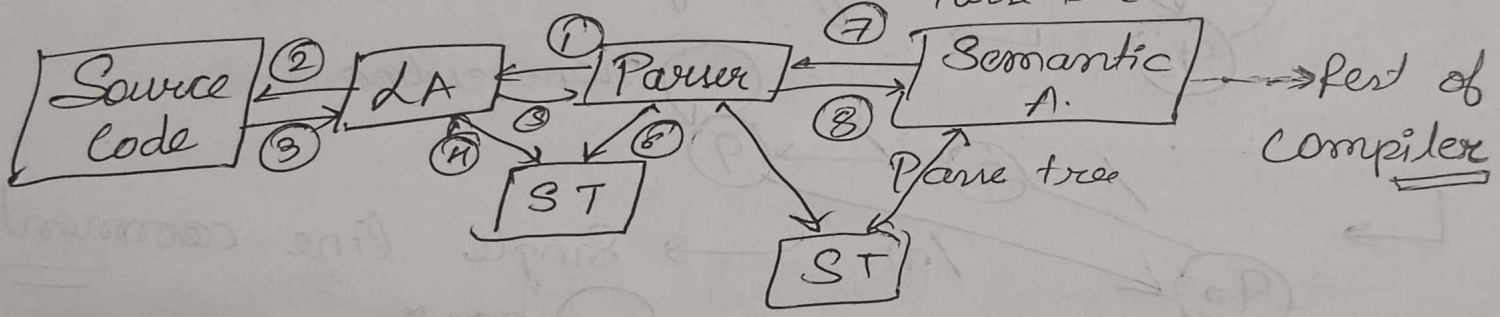
Lex

C/w

9/02/26

Syntax Analysis

↳ we have to generate parse tree.



We have to generate parse Tree.

↳ from Context free grammar.

↳ [CNF] → should be there

as this will grow horizontally

↳ Binary parse tree hoga isse hence we use CNF

↳ It will be easier to handle precedence

Error handling in Syntax Analysis
↳ Panic mode recovery during error.

Error :-

Phrase Elimination → works on prefix only
↳ insertion
↳ updation
↳ deletion.

Panic mode recovery
↳ It Performs Successive deletion until well formed token isn't received

Global correction
↳ theoretical concept
↳ high computational cost

↳ Parse tree
↳ L

Context free grammar

Que Write down a context free grammar for lang L, in which all the strings over a and b are palindrome

$$S \rightarrow aSa \mid bSb \mid \epsilon \mid a \mid b$$

↳ If a and b are in equal no then ↳ $S \rightarrow aSa \mid bSb \mid SS$

Ques Generate CFL for lang over abcd

$$a^{2n+3} b^{3m+2} c^{2m+1} . D^{n+2} \quad n, m > 0$$

$S \rightarrow aaS_1dd \mid$
$S_1 \rightarrow bbbS_2cc \mid aaaS_1dd \mid S$
$S_2 \rightarrow bbc \mid \epsilon$

$S \Rightarrow aaS_1d$
$S_1 \rightarrow S \mid aaaS_2dd$
$S_2 \rightarrow bbbS_2cc$ bbbS$_3$cc
$S_3 \rightarrow bbc$

Parse - Tree

All the leaf nodes will be terminal only.

Derivation is the process of generating a derived string by replacing variables using any one of the given production rules. from starting symbol

↳ Sentinel form.

→ we decide ~~have to know about~~ two things

↳ which variable needs to be replaced
which production rules has to be used for derivation

→ two forms of parse tree

↳ leftmost derivation
↳ right most derivation.

$L = \{a^m b^{2n} c^m . d^{m+1} / m, n \geq 0\}$

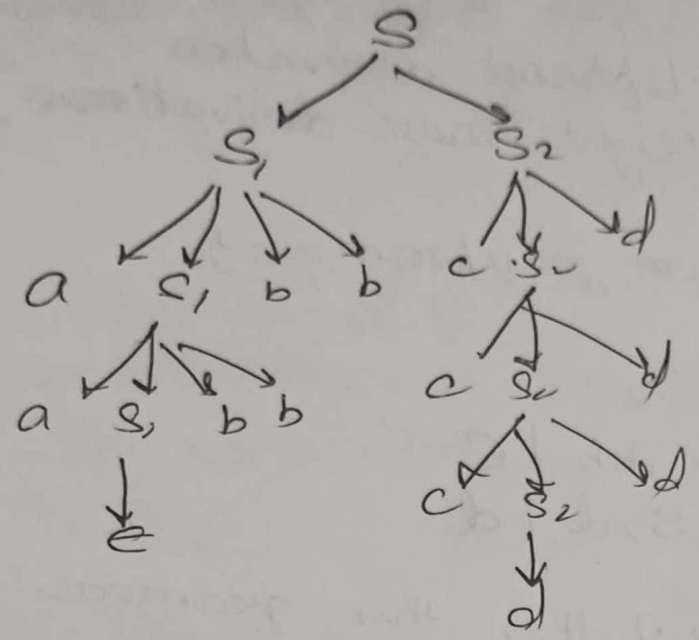
$S \rightarrow S_1 S_2$
$S_1 \rightarrow aS_1bb \mid \epsilon$
$S_2 \rightarrow cS_2d \mid d$

To check whether this grammar is ambiguous.

↳ i/p ⇒ $a^2b^4c^3d^4$

Let's do leftmost derivation

$S \rightarrow S_1 S_2$
↓
aS_1bbS_2
↓
aaS_1bbbbS_2
↓
$aabbbbS_2$
↓
$a^2b^4CS_2d$
↓
$a^2b^4CSS_2dd$
↓
$a^2b^4CCCS_2ddd$
↓
$a^2b^4CCCdddd$



RMD

$S \rightarrow S_1 S_2$

$S_1 c S_2 d$

$S_1 cc S_2 dd$

$S_1 ccc S_2 ddd$

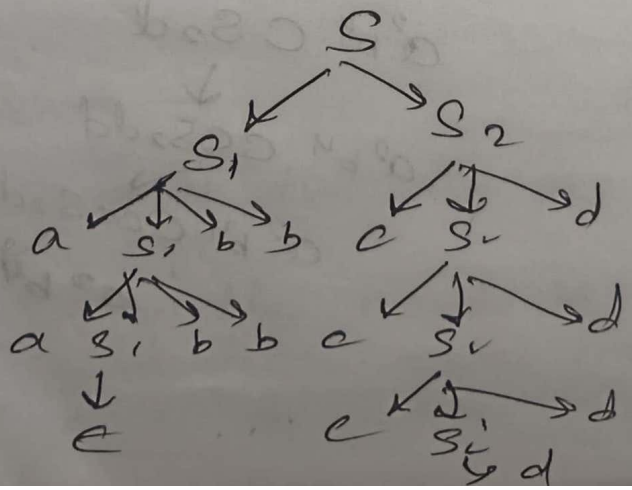
$S_1 c^3 d^4$

$a S_1 b^2 c^3 d^4$

$aa S_1 b b b b c^3 d^4$

$aa b b b b c^3 d^4$

Parse tree



Parse tree is same for both hence our grammar is non ambiguous

Ques

$S \rightarrow SS+$ → (Postfix notation)

$S \rightarrow SS+ \mid SS* \mid a$

try for given string → aa+a*

check ambiguous or not

aa+a*

LMD

$S \Rightarrow SS*$

$S \rightarrow SS*$

$\rightarrow SS+S*$

$aS+S*$

$aa+S*$

$aa+a*$

RMD

$S \rightarrow SS*$

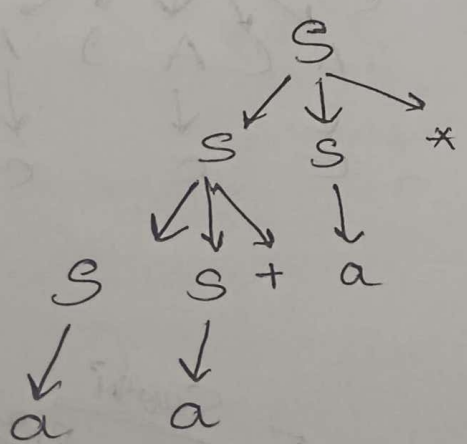
$Sa*$

$SS+a*$

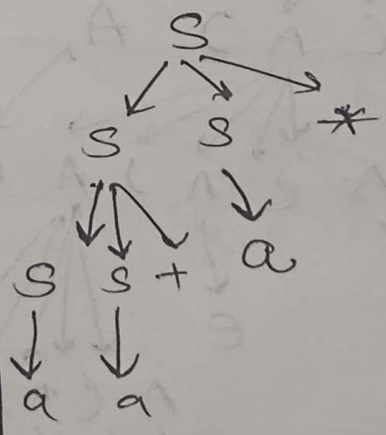
$Sa+a*$

$aa+a*$

Parse tree



Parse Tree



Unambiguous

Ques $A \rightarrow A(A)A \mid \in$

String → (()())

→ total four Parse trees → where they all are not the same → so Ambiguous

LMD

$A \rightarrow A(A)A$

$\in(A)A$

$(A(A)A)A$

$((A)A)A$

$\in((A)A)A$

$(()A(A)A)A$

$\in(()(A)A)A$

$(()()A)A$

$(()())A$

$(()())$

2 Parse trees based on choice

PMD

$A \rightarrow A(A)A$

$A(A)\in$

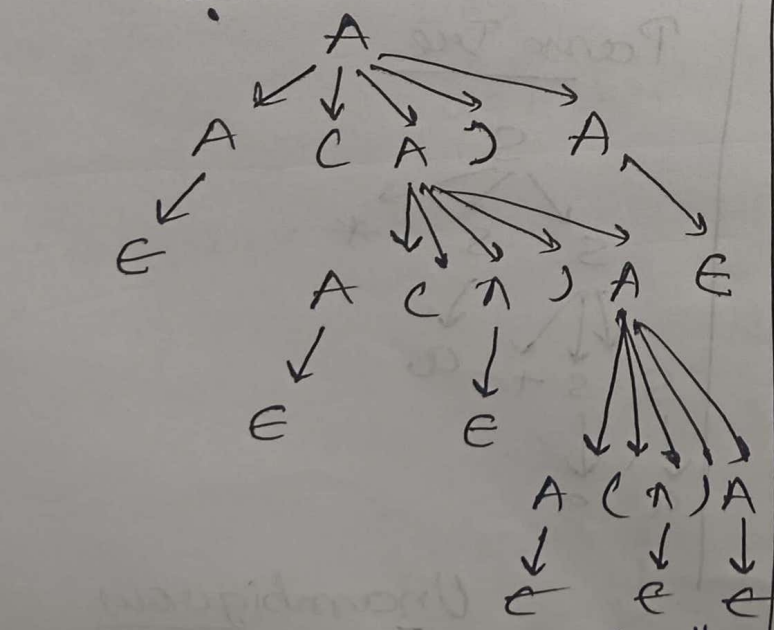
$A(A(A)A)$

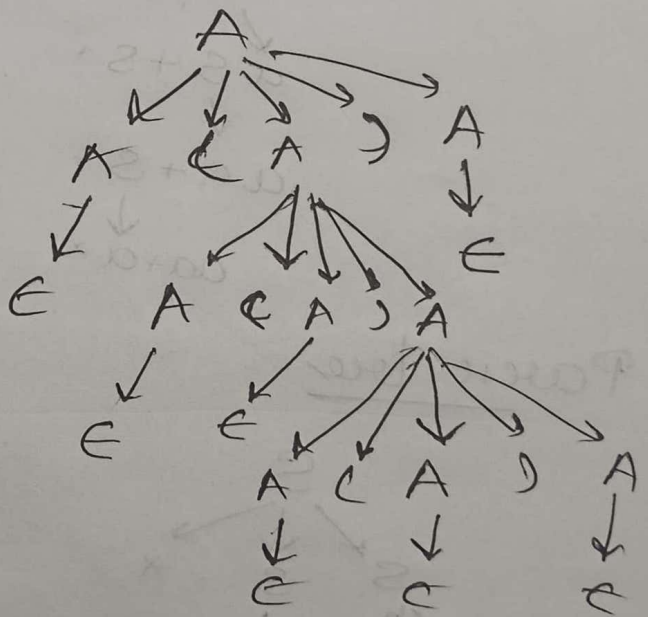
$A(A(A)\in(A)A)$

$A(A(A)A(A))\in$

$\in\ \in\ \in\ \in\ \in$

$(()())$





Sushi Prachi

Aprajita tum bohot acchi ho alelelele.

Any two parse trees can be shown diff to show ambiguity.

Ques

$S \rightarrow aSbS \mid bSaS \mid \epsilon$

String → aa bbab

↳ LMD

$S \rightarrow aSbS$

$\rightarrow aaSbSbS$

$\rightarrow aa\epsilon bSbS$

$\rightarrow aabSbS$

$\rightarrow aab\epsilon Sbs$

$\rightarrow aabbaSbS$

$\rightarrow aabb\epsilon SbS$

$\rightarrow aabbabS$

$\rightarrow aabbab\epsilon$

$\rightarrow aabbab$

RMD

$S \rightarrow aSbS$

$\rightarrow aSbaSbS$

$\rightarrow aSbaSb\epsilon$

$\rightarrow aSbaSb\epsilon$

$\rightarrow aSbab$

$\rightarrow aaSbSbab$

$\rightarrow aa\epsilon bSbab$

$\rightarrow aa\epsilon b\epsilon bab$

$\rightarrow aabbab$

$S \rightarrow aSbS$

$\rightarrow aSbaSbS$

$\rightarrow aSba\epsilon bS$

$\rightarrow aSbabS$

$\rightarrow aSbab\epsilon$

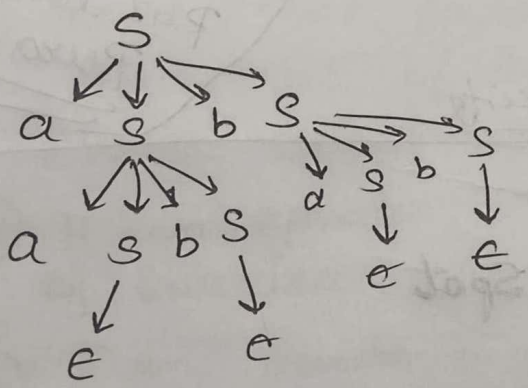
$\rightarrow aSbab$

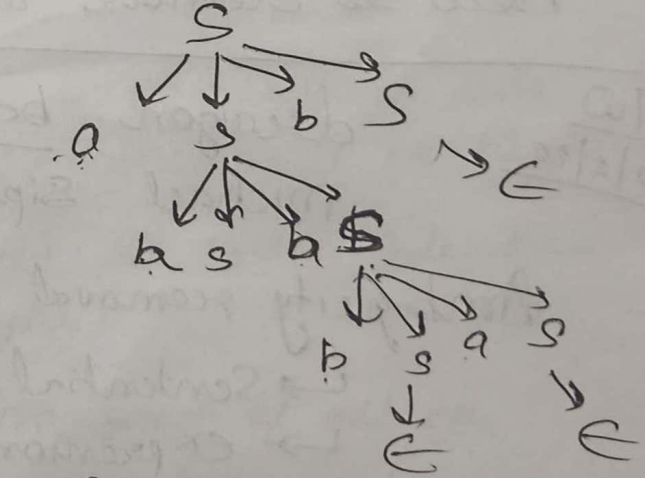
$\rightarrow aaSbSbab$

$\rightarrow aa\epsilon bSbab$

$\rightarrow aa\epsilon b\epsilon bab$

$\rightarrow aabbab$





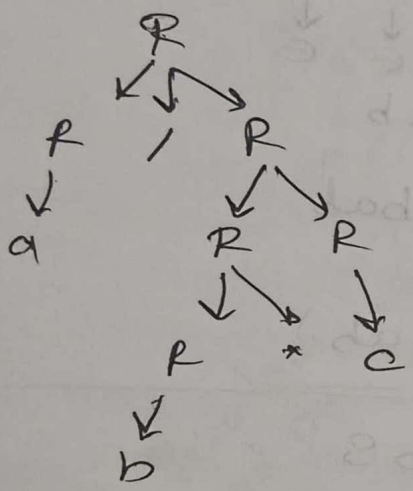
Ambiguous

Ques

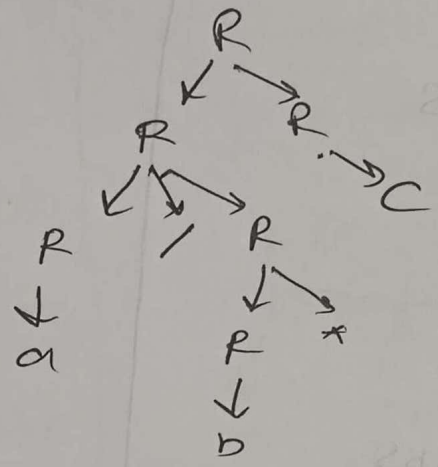
$R \rightarrow R/R/RR/R^*/(R)/a/b/c$ (/ : division)

String a/b*c

$R \rightarrow R/R$
$R \rightarrow R/RR$
$R \rightarrow R/R^*R$
$R \rightarrow a/b^*c$



$R \rightarrow R/R$
$R \rightarrow R/RR$
$R \rightarrow R/R^*R$



ambiguous

Context free grammar
LMD, RMD
Ambiguity checking
How to remove ambiguity

→ Associativity
Precedence
Left long
Pure

C|D
10/2/26

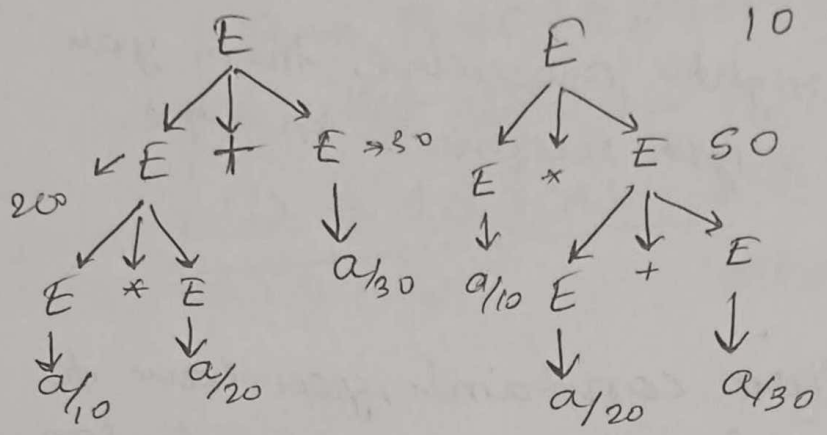
dragon book
Michael Sipser / Spot

Ambiguity removal
↳ Sentential form se removal
↳ Expression se removal

Why need to remove

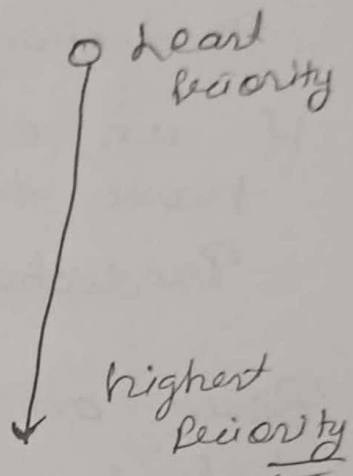
$E \to E+E \mid E*E \mid a$

$a*a+b$



$a_1 * a_2 + a_3$

10 20 30



Node represents priority of that particular operator

⇒

Ambiguity removal using operator precedence and Associativity

Rules for the precedence constraint

1. In a given parse tree, if an operator is present in any internal node at any level, that particular level will give the precedence of that operator for the entire calculation.
2. If an operator is present at bottom side of parse tree then precedence will be higher
3. If an operator is present in upper side then precedence will be higher
4. You have to perform arrangement of all the operator, so that precedence among operator is maintained.

Associativity constraints

- If an operator is left associative then you have to induce left recursion in the Production.
- If an operator is right associative then you have to induce right recursion in the Production.

Based on above two constraints, you have to introduce variables in the given production to maintain above two constraints.

$A \rightarrow A*A \mid A/A$
$A \rightarrow A+A \mid A-A$
$A \rightarrow A \wedge A$
$A \rightarrow (A) \mid id$

① take string
② check Ambiguity
③ Remove
④ Recheck

Identify terminals

↳ *, +, -, id, (,), ^, /

Arrange operators in decreasing order of their Precedence

↳ id, (), ^, *, /, + and -.

Decide levels of precedence

Level 1 → + and - ⟶ A ⟶ LR
Level 2 → * and / ⟶ B ⟶ LR
Level 3 → ^ ⟶ C ⟶ RL
Level 4 → id and () ⟶ D ⟶ —

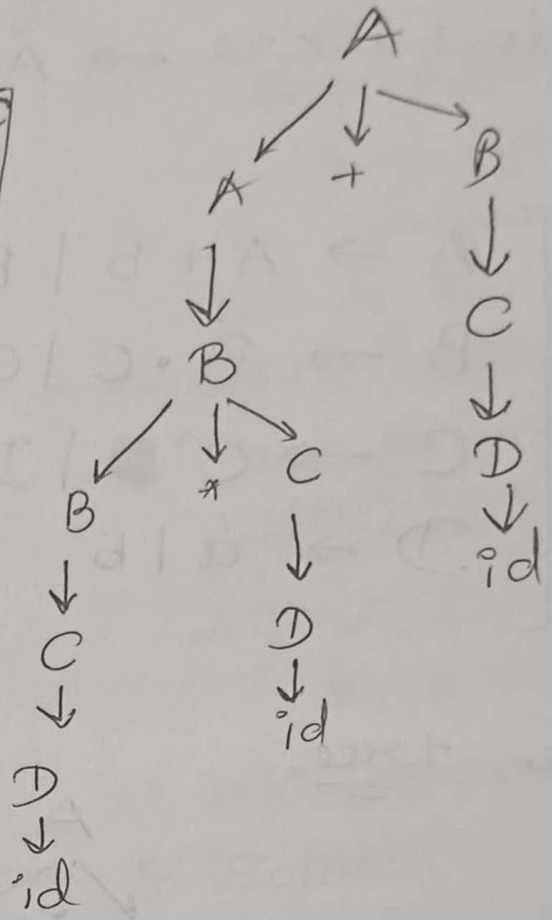
existing variables will be assigned to lower levels then make new levels

$A \rightarrow A+B \mid A-B \mid B$

$B \rightarrow B*C \mid B/C \mid C$

$C \rightarrow D \hat{} C \mid D$

$D \rightarrow id \mid (A)$



Ques ~~following the grammar~~

$A \rightarrow A+A \mid A \cdot A \mid A^* \mid a \mid b$

$+$ → Or operator

$\cdot$ → Concatenation

terminals → a, b, +, ·, *

Priority → <a and b> → *, ·, + → level 1

levels → + → level 2

· → level 3

<a and b> → level 4

* → level 3

· → level 2

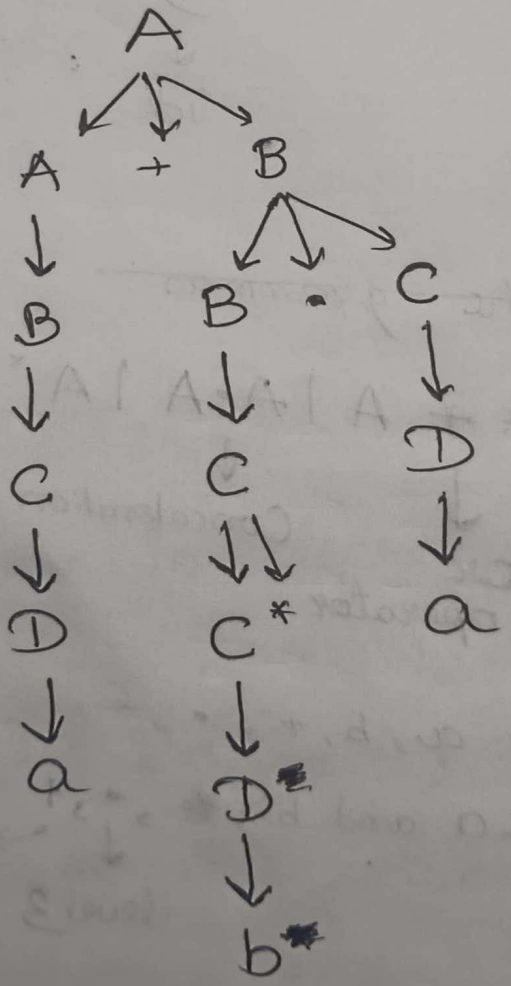
+ → level 1

level 4 <a and b> → D → —
level 3 <*> → C → L-R
level 2 <.> → B → L-R
level 1 <+> → A → L-R

$$\begin{bmatrix} A \to A+B \mid B \\ B \to B \cdot C \mid C \\ C \to C^* \mid D \\ D \to a \mid b \end{bmatrix}$$

$a + b^* \cdot a$

Parse tree.



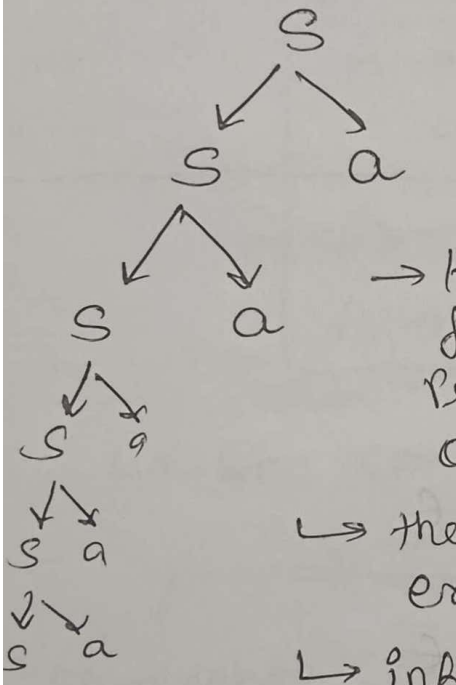
6/2/26

Parser → top down parser
↳ Bottom up parser

$S \to Sa \mid b \mid aB$ →

90% chance that we will induce left recursion in the grammar because of the rule in ambiguity removal. as most op are left associative

↳ If we don't know which rule to follow to get our string
↳ try out all possible production



→ taking first production only
↳ there is no ending
↳ infinite loop due to left recursion.

$A \to A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \ldots \mid A\alpha_2 \mid \beta_1, \beta_2) \ldots \mid \beta_n$

↳ left recursion
↳ To Solution :- convert it to right recursion

~~$A \to \beta_1 \mid \beta_2 \mid \beta_3 \mid \ldots \mid \beta_n$~~

$A \to \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \beta_4 . A' \mid \beta_n$

$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \ldots \mid \alpha_n A' \mid \epsilon$

Eg :- $E \to E + T \mid T$ (A: E, α: $+T$, β: T) convert →

$E \to TE'$
$E' \to +TE' \mid \epsilon$
$T' \to \epsilon$

Ques ① $S \to S(S)S \mid \epsilon$

② $S \to SS \mid aSb \mid bSa \mid \epsilon$

③ $E \to E+T \mid T$
$T \to T*F \mid F$
$F \to F-G \mid I$
$I \to (id) \mid E$

① $S \to \overbrace{S}^{A}\overbrace{(S)S}^{\alpha} \mid \overbrace{\epsilon}^{B}$

$S \to AA'$

$A' \to (S)SA' \mid \epsilon$ ✓

② $S \to \overbrace{SS}^{A\alpha} \mid \overbrace{aSb}^{\beta_1} \mid \overbrace{bSa}^{\beta_2} \mid \overbrace{\epsilon}^{\beta_3}$

$S \to aSbA' \mid bSaA' \mid A'$
$A' \to SA' \mid \epsilon$

③ $E \to \overbrace{E}^{A}\overbrace{+T}^{\alpha} \mid \overbrace{T}^{B}$

$T \to T*F \mid F$
$F \to F-G \mid I$
$I \to (id) \mid E$

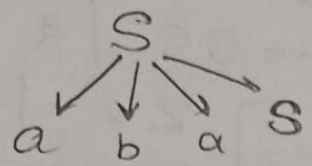
→

$E \to TE'$
$E' \to +TE' \mid \epsilon$
$T \to FT'$
$T' \to *FT' \mid \epsilon$
$F \to IF'$
$F' \to -GF' \mid \epsilon$
$I \to (id) \mid E$

Ambiguity remove → left recursion
↳ then remove left recursion
↳

$S \rightarrow abaS / abaA / abaC$

$P_1 \quad P_2 \quad P_3$



Left factoring (not suitable)

↳ Whenever there is ~~multiple~~ common factor occuring in left side of the Production rule.

↳ here aba

~~$A \rightarrow AY_1 / AY_2 / AY_3 / \ldots / AY_m$~~

$A \rightarrow \gamma X_1 / \gamma X_2 / \gamma X_3 / \ldots \gamma X_m$

γ is common

$B \rightarrow X_1 / X_2 / X_3 / \ldots / X_m$

$A \rightarrow \gamma B$

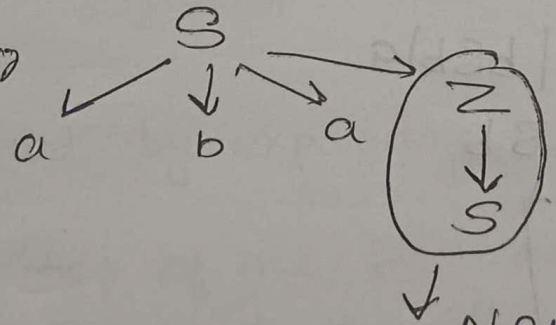
↳ So converting

$S \rightarrow abaS / abaA / abaC$

↓

$Z \rightarrow S / A / C$

Elimination of left factoring $S \rightarrow abaZ / q$



or A or C

↓

Now backtracking is restricted to this Symbol only

Ques $S \to \overset{\alpha_1}{SS+} \mid \overset{\alpha_2}{SS*} \mid \overset{\beta_1}{a}$

$S \to aS'$

$S' \to S+S' \mid S*S' \mid \epsilon$

$S \to SSZ \mid a$

$Z \to + \mid *$

Now elimination of left factoring

$S \to aS'$

$Z \to + \mid *$

$S' \to SZS' \mid \epsilon$

$S \to SSZ \mid a$

$Z \to + \mid *$

$S \to aS'$

$S' \to SZS' \mid \epsilon$

Same from both orders

Ques $A \to aAB \mid aBC \mid aAC$

$Z \to aA$

$A \to ZB \mid aBC \mid ZC$

$Z \to aA$

$A \to ZY \mid aBC$

$Y \to B \mid C$

$A \to aAZ \mid aBC$

$Z \to B \mid C$

$A \to aX$

$X \to AZ \mid BC$

$Z \to B \mid C$

Ques $S \to bSSaaS \mid bSSaSb \mid bSb \mid a$

$S \to bSSX \mid bSb \mid a$

$X \to aaS \mid aSb$

$S \to bSZ \mid a$

$Z \to SX \mid b$

$X \to aY$

$Y \to aS \mid Sb$

Top down parser

- With Backtracking → most Powerful
- Non-backtracking → no question
  - Recursive Descend Parser
    - Sbse Bkdaas
  - Table driven Parser (numerical specific)
    - Can predict which production to use

LL1 Parser
- ↳ LL1 grammar
- ↳ shift Reduce Parser
- ↳ operator - parser

First follow Parser
- ↳ tak

LAR } Parser
CLAR

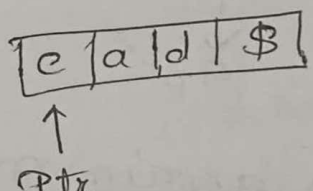
---

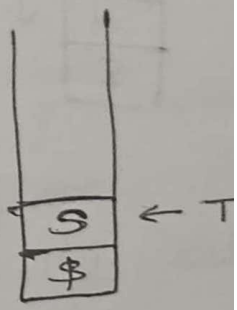
c/w
17/2/26

Parser

Backtracking parser

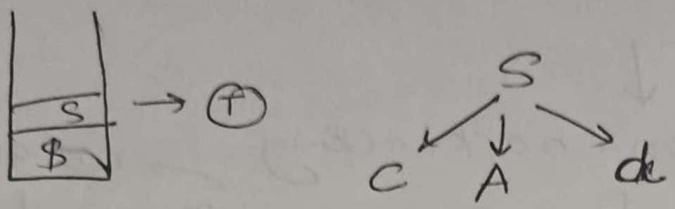
$S \rightarrow cAa$
$A \rightarrow ab/a$
$W = cad$

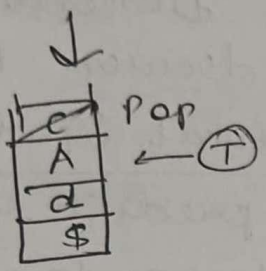


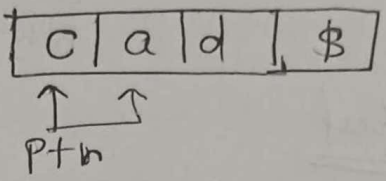


① start by expanding top of the stack

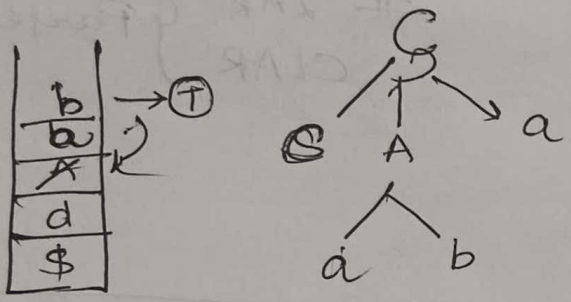
If top of the stack matches with Ptr then Pop if doesn't match then backtracking

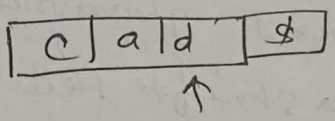




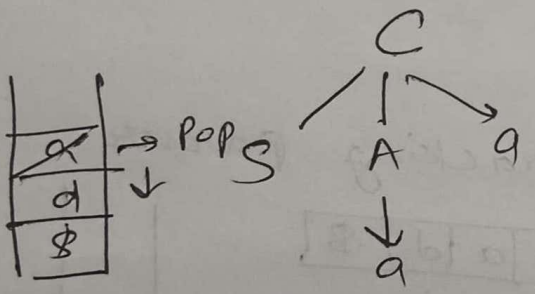


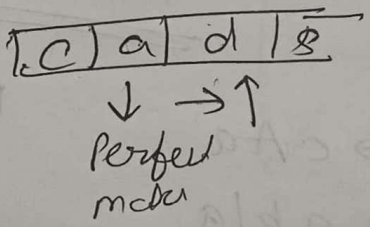
variable hai to expand





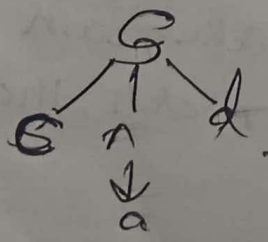
doesn't match
So backtrack





d → again match
again PoP

$ matches with $ → So end



→ this is the Parse tree

Drawback of backtracking parser

↳ time and
↳ left recursion
↳ tedious process

left factor
remove left recursion

Recursive descend. Parser

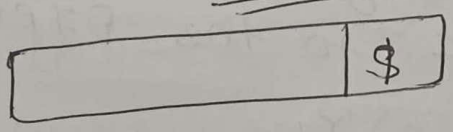
↳ Predictive parser or removing backtracking
↳ If we remove recursion
↳ non recursive Predictive Parser

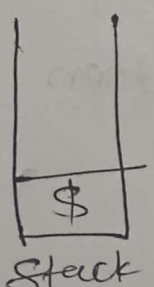
not to be studied

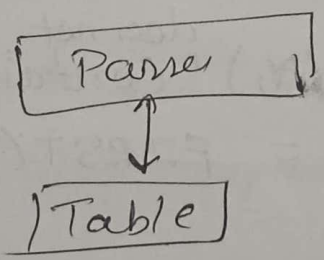
When we have multiple ~~productions~~ for single variable and we don't know which one will be used. → then there is backtracking
↳ So to remove backtracking, we

tabular parser (table driven parser)

I/P Tape







↳ LL(K) grammar
table driven
Parser only works
on this

↳ Left most Derivation

LL(k) → Look Ahead Symbol
↳ Left to right
↳ Left most Derivation

k is the no of Symbols we'll see to make decision.

First and follow set

↳ tells from a variable which terminal can be initially produced

FIRST(X) = { }
↓ ↳ Set representation
Variable

Rules

① If X is a terminal then FIRST(X) = {X}

② If X is ε or there is a production of X producing ε then add ε to the FIRST(X)

③ $X \rightarrow Y_1 Y_2 Y_3 \ldots Y_k$
If FIRST(Y_1) does not contains ε then FIRST(X) = FIRST(Y_1)

④ $X \rightarrow Y_1 \ldots Y_k$
If FIRST(Y_1) has ε then

FIRST(X) = {FIRST(Y1) - ε} ∪ FIRST(Y2 - Yk)

Ques

~~X~~ → ABC
A → a|ε
B → b
C → c

FIRST(C) = {c}
FIRST(B) = {b}
FIRST(A) = {a} ∪ {ε}
= {a, ε}

FIRST(~~X~~) = {FIRST(A) - ε}
∪ FIRST(BC)

FIRST(X) = {a} ∪ FIRST(B)
= {a} ∪ {b}
= {a, b}

Ques

X → ABC
A → a|ε
B → a|b|ε
C → a|b|c|ε

FIRST(C) = {a, b, c, ε} ✓
FIRST(B) = {a, b, ε} ✓
FIRST(A) = {a, ε} ✓
FIRST(X) = {FIRST(A) - ε} ∪ FIRST(BC)
= {a} ∪ {FIRST(B) - ε} ∪ FIRST(C)
= {a} ∪ {a, b} ∪ {a, b, c, ε}
= {a, b, c, ε} ✓

Qeep $S \Rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC \mid \epsilon$

$D \rightarrow Ef$

$E \rightarrow g \mid \epsilon \quad f \rightarrow f \mid \epsilon$

$FIRST(F) = \{f, \epsilon\}$ ✓

$FIRST(E) = \{g, \epsilon\}$ ✓

$FIRST(D) = \{g\} \cup FIRST(F)$

$FIRST(D) = \{g\} \cup \{f, \epsilon\}$

$= \{g, f, \epsilon\}$ ✓

$FIRST(C) = FIRST(b) \cup FIRST(C) \cup \epsilon$

$FIRST(C) = \{b, \epsilon\}$ ✓

$FIRST(B) = \{c\}$

$FIRST(S) = \{a\}$ ✓

ees

$S \rightarrow A\,aAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

$FIRST(B) = \epsilon$, $FIRST(A) = \epsilon$

$FIRST(S) = FIRST(AaAb) \cup FIRST(BbBa)$

$= FIRST(aAb) \cup FIRST(bBa)$

$= \{a, b\}$ ✱

Ques $E \to E+T \mid T$
$T \to T*F \mid F$
$F \to (E) \mid id$

first remove left recursion and left factoring

$E \to TE'$
$E' \to +TE' \mid \epsilon$
$T \to FT'$
$T' \to *FT' \mid \epsilon$
$F \to (E) \mid id$

No ~~left~~ factoring

First (F) = {(, id} ✓
First (E) = FIRST(T)
First (E') = {+, ε} ✓
First (T') = {*, ε} ✓
First (T) = {(, id} ✓
First (E) = {(, id} ✓

c/o
19/2/26

FOLLOW nikalta hai from production Side
→ directly remove left recursion

① If S is starting Symbol then add $ in the follow of $

FOLLOW(S) = {$}

$ will always be present in follow set and ε will always be present in FIRST SET

(2) If there have a production

$A \to \alpha B$

FOLLOW(B) =

B is rightmost variable

So FOLLOW(B) = FOLLOW(A)

(3) If $A \to \alpha B \beta$ 'B in not rightmost

FOLLOW(B) = FIRST(β) → β can be multiple

Now (i) If $\in$ part of FIRST(β)

FOLLOW(B) = {FIRST(β) − $\in$} ∪ FOLLOW(A)

(ii) If $\in$ is not part of FIRST(β)

FOLLOW(B) = FIRST(β)

Ques

$E \to E+T \mid T$

$T \to T*F \mid F$

$F \to (E) \mid id$

$E \to TE'$

$E' \to +TE' \mid \in$

$T \to FT'$

$T' \to *FT' \mid \in$

$F \to (E) \mid id$

FOLLOW(E') = FOLLOW(E)
FOLLOW(E) = {)}
FOLLOW(T') = FOLLOW(T)
FOLLOW(T) = ~~FOLLOW(E')~~ FIRST(E')
FOLLOW(F) = FIRST(T')

FOLLOW(E) = {)}
FOLLOW(F) = {*, ~~+~~,)} ∪ FOLLOW(T)
FOLLOW(T) = {+,)} ∪ FOLLOW(E)

FOLLOW(E) = {)} ∪ {$} ✓
FOLLOW(T) = {+,)} ∪ {$} ✓
FOLLOW(F) = {*, +,)} ∪ {$} ✓
FOLLOW(E') = {)} ∪ {$} ✓
FOLLOW(T') = {+,)} ∪ {$} ✓

Ques S → aS | bS | aAb | bAa
A → a | b | aBa | ∈
B → b | a | bAb | ∈

~~FIRST~~ FIRST(S) = {a, b}
FIRST(A) = {a, b, ∈}
FIRST(B) = {b, a, ∈}

FOLLOW(S) = {$}
FOLLOW(A) = {b} ∪ {a}
FOLLOW(B) = {a}

Ques

$S \rightarrow aBDh$
$B \rightarrow cC$
$C \rightarrow bC \mid \epsilon$
$D \rightarrow EF$
$E \rightarrow g \mid \epsilon$
$F \rightarrow f \mid \epsilon$

FOLLOW(S) = {$}
FOLLOW(B) = FIRST(Dh)
= {g, f} ∪ {h} ∪ FOLLOW(S)
FOLLOW(B) = {g, f, h, $}
FOLLOW(C) = {g, f, h, $}
FOLLOW(F) = FOLLOW(D) = {h}
FOLLOW(D) = {h}
FOLLOW(E) = FIRST(F)
= {f} ∪ {h}

C/W
24/2/22

How to Construct a LL1 Parser

↳ Find first and follow
↳ check if the grammar is suitable for parsing
↳ ✗ ↳ how to find out
↳ By constructing a Parsing table
↳

$A \to \alpha | \beta$

① FIRST(α) and FIRST(β) must be a disjoint set
↳ FIRST(α) ∩ FIRST(β) = ϕ.

② Either → $\alpha \to \epsilon$ or $\beta \to \epsilon$ (directly or indirectly)
but not both.

③ If $\alpha \xrightarrow{*} \epsilon$
↳ can be reduced to ϵ
then FIRST(β) ∩ FOLLOW(A) == ϕ

If $\beta \xrightarrow{*} \epsilon$, FIRST($\alpha$) ∩ FOLLOW(A) == ϕ

↳ Based on these rules check if a grammar is LL1 suitable or not.

Generalisation

① FIRST(α_1) ∩ FIRST(α_2) ∩ ... FIRST(α_3) = ϕ → $\alpha_1 \cap \alpha_2 = \phi$ as well for any

② not all should go to ϵ
↳ only one should go to ϵ.

③ ↳ FIRST($\alpha_2 \alpha_3 \alpha_4$) ∩ FOLLOW(A) == ϕ
↳ Bacha Portion and follow(A) all should be disjoint

any pair ↓ all must be disjoint

Ques

$A \to aB / aC$

$A \to \underbrace{a}_{\alpha} \underbrace{B}_{\beta} \mid \underbrace{b}_{\alpha} \underbrace{C}_{\beta}$

→ all the Productions should be checked for whether it is LL1 suitable or not.

Ques

$A \to \underbrace{a}_{\alpha} \underbrace{B}_{\beta} \mid \underbrace{CD}_{\beta}$

(1) $A \to aB \mid CD$
(2) $C \to c \mid \epsilon$
(3) $D \to d \mid \epsilon$
(4) $B \to ac \mid bc$

→ check each Production one by one

↳ If it is suitable for LL1 Parser or not.

⇒ Let's make Parse table → not ϵ

$A \to aB / CD$ (1)
$C \to c / \epsilon$ (2)
$D \to d / \epsilon$ (3)
$B \to ac / bc$ (4)

	a	b	e	d	$
A					
B					
C					
D					

$ → Special Symbol for end of stack

↳ find first and follow of all

First(A) = {a, c, d, ϵ}
Follow(A) = $

FIRST(B) = {a, b}
Follow(B) = $

FIRST(C) = {c, ε}
FOLLOW(C) = {d, $}

FIRST(D) = {d, ε}
FOLLOW(D) = {$}

Now fill the table

	a	b	c	d	$
{a, c, d, ε} A	A → aB	–	A → CD	A → CD	A → CD
{a, b, ε} B	B → aC	B → bC	–	–	–
{c, ε} C	–	–	C → ε	–	C → ε
{d, ε} D	–	–	–	D → d	D → ε

Ques 2S/2/2G

S → aBDh
B → cC
C → bC | ε
D → EF
E → g | ε
f → f | ε

If Parsing table has multiple entries then not suitable for LL1 Parsing.

↳ Let's create Parsing table

	a	b	c	g	f	h	$
S	S → aBDh						
B			B → cC				
C		C → bC		C → ε	C → ε	C → ε	
D				D → EF	D → EF	D → EF	
E				E → g	E → ε	E → ε	
F					f → f	f → ε	

Table filling Rules

α means table

① Add $A \to \alpha$ to $\alpha[A, a]$

If $a \in FIRST(\alpha)$

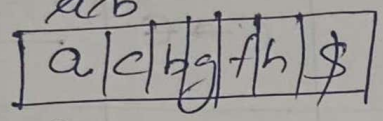
② Add $A \to \alpha$ to $\alpha[A, a]$

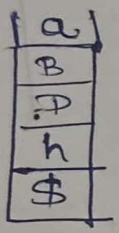
If $a \in FOLLOW(A)$ then

If $\in$ is part of $FIRST(\alpha)$

Now Parsing

String = abdb





Stack

LL(1) → top down parser

α

→ Just keep moving the two pointers and check with the Parsing table

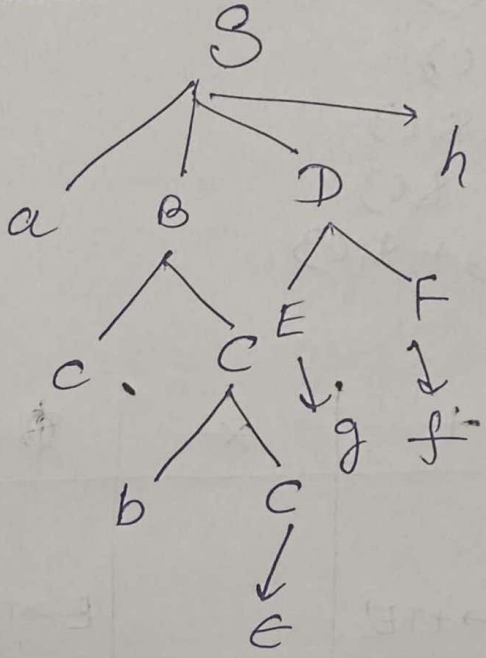
↳ Place Productions using the Parsing table.

Way of writing

Stack	Input	Action
$S	a c b g f h $	S → a B D h
$ h D B a	a c b g f h $	POP a
$ h D B	c b g f h $	B → c C
$ h D C c	c b g f h $	Pop c
$ h D C	b g f h $	C → b C

Stack	Input	Action
$hDcb	bgfh$	Pop b
$hDc	gfh$	C → ∈
$hDG	gfh$	D → Ef
$hD	gfh$	E → g
$hFE	gfh$	Pop g → Perfect match
$hfg	gfh$	Pop f
$hf	fh$	Pop h
$h	h$	
$	$	

Now, Constructing Parse tree



Ques $E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid f$
$F \rightarrow (E) \mid id$

$id \rightarrow id*id$

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \epsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$
$id \rightarrow id$
$id' \rightarrow *id \mid \epsilon$

$E \to TE'$
$E' \to +TE' \mid \epsilon$
$T \to FT'$
$T' \to *FT' \mid \epsilon$
$f \to (E) \mid id$

FIRST(E') = {+, ε}
FIRST(F) = {(} ∪ {id}
FIRST(T') = {*, ε}
FIRST(T) = {(}
FIRST(E) = {(}

FOLLOW(E) = {$,)}
FOLLOW(E') = FOLLOW(E) = {$,)}
FOLLOW(F) = {*, } ∪ FOLLOW(T)
FOLLOW(T') = FOLLOW(T) = {+, $,)}
FOLLOW(T) = {+, } ∪ FOLLOW(E)
= {+, $,)}

FOLLOW(E) = {$,)}
FOLLOW(E') = {$,)}
FOLLOW(T) = {+, $,)}
FOLLOW(T') = {+, $,)}
FOLLOW(F) = {*, +, $,)}

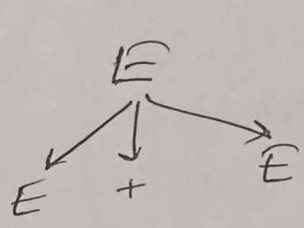
	(	+	*	id	$	)
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to (E)$ id			$F \to (E)$		

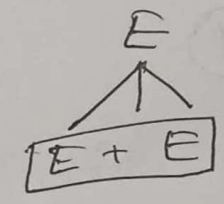
Bottom up parser

- Shift-Reduce Parser
- LR Parser
  - → Left to right Scanning and right most derivation in reverse order.
  - LR(0)
  - SLR(1)
  - LALR(1)
  - CLR(1)
  - LR(0)
  - LR(1)
  - → there does work on unambiguous grammar
- Operator Precedence Parser

LALR → Look Ahead LR
CLR → Canonical LR

$E \to E+E \mid E-E$
$E \to (E) \mid id$
$id \to id-id$

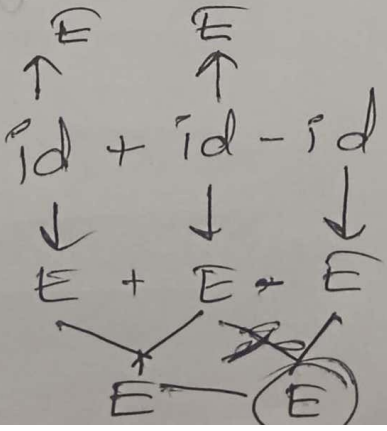


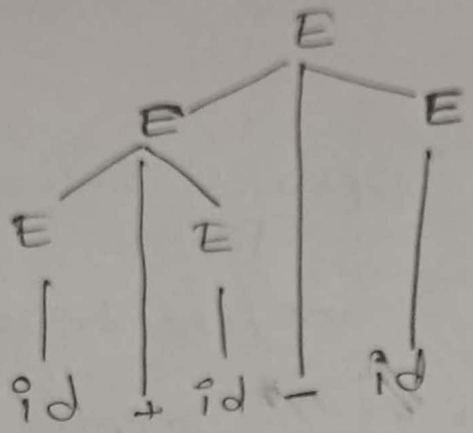


→ handle
↘ Combination of variables and terminal

Our task → identify handle and replace it with some Production.

handle - the substring that matches right hand side of any production.





→ Rightmost derivation in reverse order

Shift Reduce Parser (Mid Syllabus upto this)

↓

least Powerful Parser

Shift (1)
Reduce (2)
Accept (3)
Reject (4)

Stack	Input	Action

* it might happen that the string is not part of grammar

* Conflicts

(1) SR conflict → Shift Reduce conflict

(2) RR conflict → give priority to Shift

↓

First remove ambiguity

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

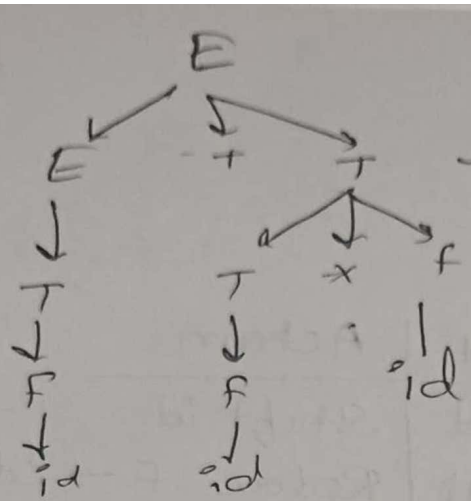
$F \rightarrow (E) \mid id$

idhar se match

Stack	Input	Action
$	id+id*id$	Shift id
$id	+id*id$	Reduce $F \rightarrow id$
$F	+id*id$	Reduce $T \rightarrow F$
$T	+id*id$	Reduce $E \rightarrow T$
$E	+id*id$	Shift +
$E+	id*id$	Shift id
$E+id	*id$	Reduce $F \rightarrow id$
$E+F	*id$	Reduce $T \rightarrow F$
$E+T	*id$	Shift *
$E+T*	id$	Shift id
$E+T*id	$	Reduce $F \rightarrow id$
$E+T*F	$	Reduce $T \rightarrow T*F$
$E+T	$	Reduce $E \rightarrow E+T$
$E	$	Acceptance

Conflict between shift and reduce

when na shift karne ka koi such hai na reduce karne ka koi hai and symbols pche nhi hai then Rejection



→ lower precedence

→ higher Precedence

[Parse tree is Right most derivation]

Ques

$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

W = "abbcde"

Stack	Input	Action
$	abbcde	Shift a
$a	bbcde	Shift b
$ab	bcde	Reduce A→b
$aA	bcde	Shift b
$aAb	cde	Shift c → conflict
~~$aAb~~	~~cde~~	~~Reduce A→b~~
$aAbc	de	Reduce A→Abc
~~$aAA~~	~~cde~~	~~Shift c~~
~~$aAAc~~	~~de~~	~~Shift d~~
~~$aAAcd~~	~~e~~	~~Reduce B→d~~
~~$aAAeB~~		

Shifted (c)

$aA	de	shift d
$aAd	e	Reduce B→d
$aAB	e	Shift e
$aABe	$	Reduce S→aABe
$S	$	Accepted

Ques

$E \rightarrow 2E2$
$E \rightarrow 3E3$
$E \rightarrow 4$

String
$W = 33244233$

Stack	Input	Action
$	33244233	Shift 3
$3	3244233	Shift 3
$33	244233	Shift 2
$332	44233	Shift 4
$3324	4233	Reduce E→4 → Shift or Reduce
$332E	4233	Shift 4 → Shift or Reduce
$332E4	233	Reduce E→4 → Shift or Reduce
$332EE	233	Shift 2
$332EE2	33	Shift 3
$332EE23	3	Shift 3
$332EE232	$	Rejected

↳ In case there is a conflict and I chose shift, then show using Reduce also

⇒ Lexical Analysis
Compiler structure → output of phase
↳ ✗

Phases → Frontend
Backend

Analysis
Synthesis
Symbol table

Lexical Analysis → Lexim generation
Token identification
↳

Syntax Analysis
↳ Bootstrapping
↳ Ambiguity → Removal
↳ check
↳ Parse tree
↳ LMD, RMD
↳ Ambiguity → Removal
↳ then left Recursion
↳ then left Recursion
Removal
then left factoring
Removal

↳ Parser
↳ Backtracking
↳ LL1
↳ first follow
↳ Shift Reduce Parser